

модели для рекомендаций

Отличный вопрос! Разберем каждый бустинг по порядку, с акцентом на то, **когда и для чего использовать**.

1. XGBoost (eXtreme Gradient Boosting)

Как работает:

- ```
Философия: "Оптимизация всего"
```
1. Gradient Boosting + регуляризация (L1/L2)
  2. Использует вторые производные (гессианы) для более точного обновления весов
  3. Параллельное построение деревьев (по уровням)
  4. Встроенная обработка пропущенных значений (sparsity-aware split)
  5. Прунинг деревьев (max\_depth, gamma, min\_child\_weight)

### Особенности:

- **Регуляризация** - защита от переобучения
- **Блочная структура** - данные разбиваются на блоки для быстрого поиска сплитов
- **Кэширование** - предвычисление статистик
- **Weighted Quantile Sketch** - эффективный поиск лучших сплитов

### Ключевые параметры:

```
params = {
 'objective': 'binary:logistic', # или 'reg:squarederror', 'rank:pairwise'
 'n_estimators': 100,
 'max_depth': 6,
 'learning_rate': 0.3,
 'subsample': 0.8, # случайные подвыборки
 'colsample_bytree': 0.8, # случайные признаки
 'reg_alpha': 1, # L1 регуляризация
 'reg_lambda': 1, # L2 регуляризация
 'gamma': 0, # минимальное снижение loss для сплита
 'min_child_weight': 1 # минимальная сумма весов в листе
}
```

### Когда использовать XGBoost:

- ✓ **Использовать когда:**

- Нужна **максимальная точность** (часто выигрывает соревнования)
- Данные **структурированные/табличные**
- Есть время на **тонкую настройку** гиперпараметров
- Нужна **стабильность и воспроизводимость**
- **Средние/большие** датасеты (до 10M строк)

### ❌ Не использовать когда:

- Очень большие данные (сотни миллионов строк) - медленно
- Нужны **быстрые итерации** (долго обучается)
- **Категориальные признаки** с высокой кардинальностью (нужна предобработка)

---

## 2. LightGBM (Light Gradient Boosting Machine)

### Как работает:

```
Философия: "Скорость и эффективность на больших данных"
1. Gradient-based One-Side Sampling (GOSS) -
 берет все примеры с большим градиентом, случайную выборку с малым

2. Exclusive Feature Bundling (EFB) -
 объединяет редкие категориальные признаки

3. Leaf-wise growth (а не level-wise как XGBoost)
 Растет вглубь, а не вширь
```

### Особенности:

- В 20-100 раз быстрее XGBoost на больших данных
- Меньше потребление памяти
- Поддержка GPU (очень эффективная)
- Встроенная обработка категориальных (без one-hot)

### Ключевые параметры:

```
params = {
 'objective': 'binary',
 'boosting_type': 'gbdt', # или 'dart', 'goss'
 'num_leaves': 31, # ВАЖНО: вместо max_depth
 'learning_rate': 0.1,
```

```
'feature_fraction': 0.8, # аналог colsample_bytree
'bagging_fraction': 0.8, # аналог subsample
'bagging_freq': 5, # частота бэггинга
'min_data_in_leaf': 20, # аналог min_child_weight
'categorical_feature': ['cat_col1', 'cat_col2'] # Указываем
категориальные!
}
```

## Когда использовать LightGBM:

### ✓ Использовать когда:

- Очень большие датасеты (миллионы/миллиарды строк)
- Нужна максимальная скорость обучения
- Много категориальных признаков с высокой кардинальностью
- Ограниченные ресурсы памяти
- Задачи real-time предсказаний (быстрый инференс)
- Learning to Rank задачи (отличная реализация)

### ✗ Не использовать когда:

- Очень маленькие датасеты (< 10k строк) - может переобучиться
- Нужна максимальная интерпретируемость (листовые деревья сложнее)
- Критически важна максимальная точность (XGBoost иногда точнее)

---

## 3. CatBoost (Categorical Boosting)

### Как работает:

# Философия: "Лучшая обработка категориальных признаков"

1. Ordered Boosting - защита от target leakage

Использует перестановки для вычисления градиентов

2. Обязательная обработка категориальных:

- One-hot для малой кардинальности
- Target Encoding (ordered) для высокой кардинальности
- Объединение редких категорий

3. Symmetric деревья - сбалансированные, быстрый инференс

### Особенности:

- **Автоматическая обработка категорий** - главное преимущество!
- **Защита от переобучения** на категориальных (ordered boosting)
- Хорошо работает "из коробки" - меньше нужна настройка
- **Быстрые предсказания** (symmetric trees)

## Ключевые параметры:

```
params = {
 'loss_function': 'Logloss', # или 'RMSE', 'MultiClass'
 'iterations': 1000,
 'learning_rate': 0.03,
 'depth': 6, # глубина деревьев
 'l2_leaf_reg': 3, # регуляризация
 'random_strength': 1, # случайность при выборе сплитов
 'cat_features': cat_features_indices, # индексы категориальных
 'bootstrap_type': 'Bayesian', # или 'Bernoulli', 'MVS'
 'od_type': 'Iter', # overfitting detector
 'od_wait': 50 # остановка если нет улучшений
}
```

## Когда использовать CatBoost:

### ✅ Использовать когда:

- **Много категориальных признаков** (главный кейс!)
- Нужно **минимум feature engineering**
- Хочется **быстро получить хороший бейзлайн**
- Данные с **временными рядами** (ordered boosting защищает)
- Задачи с **небалансированными классами**
- Нужны **надежные доверительные интервалы**

### ❌ Не использовать когда:

- Нет категориальных признаков (лучше LightGBM/XGBoost)
- Критически важна максимальная скорость (медленнее LightGBM)
- Очень большие данные (может быть прожорлив по памяти)

## 4. AdaBoost (Adaptive Boosting)

### Как работает:

```

Философия: "Усиление слабых классификаторов"
1. Итеративно обучает слабые классификаторы (чаще всего shallow trees)
2. Увеличивает вес misclassified примеров на каждой итерации
3. Комбинирует через взвешенное голосование

Псевдокод:
weights = [1/N] * N # равные веса для всех примеров
for t in range(T):
 # Обучаем слабый классификатор с учетом весов
 classifier = train_weak_learner(X, y, weights)

 # Вычисляем ошибку
 error = sum(weights[i] for i where classifier wrong) / sum(weights)

 # Вычисляем вес для этого классификатора
 alpha = 0.5 * log((1 - error) / error)

 # Обновляем веса примеров
 for i in range(N):
 if classifier wrong on i:
 weights[i] *= exp(alpha)
 else:
 weights[i] *= exp(-alpha)

 # Нормализуем веса
 weights = weights / sum(weights)

```

## Особенности:

- Простой и интерпретируемый
- Меньше склонен к переобучению чем Gradient Boosting
- Хорош для слабых классификаторов (пеньки решений)
- Чувствителен к выбросам (увеличивает их вес)

## Ключевые параметры:

```

params = {
 'n_estimators': 50,
 'learning_rate': 1.0, # вес каждого классификатора
 'algorithm': 'SAMME.R', # или 'SAMME'
 'base_estimator': DecisionTreeClassifier(max_depth=1) # слабый
 классификатор
}

```

# Когда использовать AdaBoost:

 **Использовать когда:**

- Нужна простота и интерпретируемость
- Бинарная классификация
- Относительно небольшие датасеты
- Хочется понять основы бустинга
- Нужен быстрый бейзлайн

 **Не использовать когда:**

- Нужна максимальная точность (уступает Gradient Boosting)
- Большие датасеты (медленный)
- Мультикласс или регрессия (есть, но не лучшее)
- Есть много выбросов в данных

## СРАВНИТЕЛЬНАЯ ТАБЛИЦА

| Критерий                  | XGBoost        | LightGBM          | CatBoost      | AdaBoost       |
|---------------------------|----------------|-------------------|---------------|----------------|
| Скорость обучения         | Средняя        | ★★★★★             | Средняя       | Медленная      |
| Потребление памяти        | Высокое        | Низкое            | Высокое       | Среднее        |
| Точность                  | ★★★★★          | ★★★★★             | ★★★★★         | ★★★            |
| Категориальные признаки   | Нужен encoding | Хорошая поддержка | ★★★★★         | Нужен encoding |
| Большие данные            | Средние        | ★★★★★             | Средние       | Плохо          |
| Простота использования    | Средняя        | Средняя           | Высокая       | Высокая        |
| Настройка гиперпараметров | Сложная        | Средняя           | Простая       | Простая        |
| Интерпретируемость        | Средняя        | Средняя           | Средняя       | Высокая        |
| Защита от переобучения    | Хорошая        | Хорошая           | Очень хорошая | Средняя        |
| GPU поддержка             | Есть           | ★★★★★             | Есть          | Нет            |

# ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

## Для соревнований (Kaggle):

1. **Старт:** LightGBM (быстрая итерация, хорошая точность)
2. **Тонкая настройка:** XGBoost (максимальная точность)
3. **Если много категорий:** CatBoost
4. **Ансамбли:** Все три вместе!

## Для production:

```
Выбор зависит от требований:
if data_size > 10M_rows:
 use = "LightGBM" # скорость и память
elif categorical_features_ratio > 0.3:
 use = "CatBoost" # автоматическая обработка категорий
elif need_max_accuracy and have_resources:
 use = "XGBoost" # максимальная точность
elif need_simple_solution:
 use = "CatBoost" # работает из коробки
```

## Для рекомендательных систем:

```
Learning to Rank задачи:
1. LightGBM с lambdarank - лучший выбор, оптимизирован под NDCG
2. XGBoost с pairwise/pairwise - тоже хорошо, но медленнее
3. CatBoost с YetiRank - хорош, но менее распространен

Feature importance:
Все три дают важность признаков, но интерпретация разная:
XGBoost: weight/gain/cover
LightGBM: split/gain
CatBoost: PredictionValuesChange
```

## Типичный пайплайн:

```
def train_model(X_train, y_train, X_val, y_val):
 models = {}

 # 1. Быстрый бейзлайн
 print("Training LightGBM...")
 lgb_model = lgb.LGBMClassifier(n_estimators=100, random_state=42)
 lgb_model.fit(X_train, y_train)
```

```

models['lgb'] = lgb_model

2. Если точность важнее скорости
print("Training XGBoost...")
xgb_model = xgb.XGBClassifier(n_estimators=100, random_state=42)
xgb_model.fit(X_train, y_train)
models['xgb'] = xgb_model

3. Если много категориальных
if has_categorical:
 print("Training CatBoost...")
 cat_model = cb.CatBoostClassifier(iterations=100, verbose=0)
 cat_model.fit(X_train, y_train, cat_features=cat_cols)
 models['cat'] = cat_model

4. Ансамблирование
class EnsembleModel:
 def predict_proba(self, X):
 probs = []
 for model in models.values():
 probs.append(model.predict_proba(X)[: , 1])
 return np.mean(probs, axis=0)

return EnsembleModel()

```

## Производительность на разных задачах:

| Задача                    | Рекомендация                    | Комментарий                             |
|---------------------------|---------------------------------|-----------------------------------------|
| Классификация (табличные) | LightGBM > XGBoost ≈ CatBoost   | LightGBM быстрее, точность сравнима     |
| Регрессия                 | XGBoost ≥ LightGBM > CatBoost   | XGBoost часто точнее на регрессии       |
| Learning to Rank          | LightGBM > XGBoost > CatBoost   | LightGBM оптимизирован под ранжирование |
| Временные ряды            | CatBoost > XGBoost > LightGBM   | Ordered boosting защищает от leakage    |
| Небалансированные данные  | CatBoost > LightGBM ≈ XGBoost   | CatBoost лучше обрабатывает imbalance   |
| Мало данных               | CatBoost ≈ XGBoost > LightGBM   | LightGBM может переобучаться            |
| Много категорий           | CatBoost ★ > LightGBM > XGBoost | CatBoost создан для этого               |

# ВЫВОДЫ

1. **LightGBM** - ваш **рабочий инструмент по умолчанию**. Быстро, эффективно, хорошо масштабируется.
2. **XGBoost** - когда нужна **максимальная точность** и есть время на настройку. "Золотой стандарт" соревнований.
3. **CatBoost** - **специалист по категориальным данным**. Используйте когда много категорий или хотите минимум preprocessing.
4. **AdaBoost** - **учебный алгоритм**. Полезен для понимания бустинга, но для продакшена уступает другим.

## Мой совет для соревнований по рекомендательным системам:

```
Этап 1: Разведка – LightGBM (быстро понять задачу)
Этап 2: Генерация признаков – все три параллельно
Этап 3: Тонкая настройка – XGBoost для максимальной точности
Этап 4: Ансамблирование – взвешенное среднее всех моделей
Этап 5: Постобработка – калибровка вероятностей
```

Помните: **лучшая модель - это ансамбль разных моделей**. Часто комбинация LightGBM + XGBoost + CatBoost дает прирост в 2-5% по сравнению с лучшей одиночной моделью.